



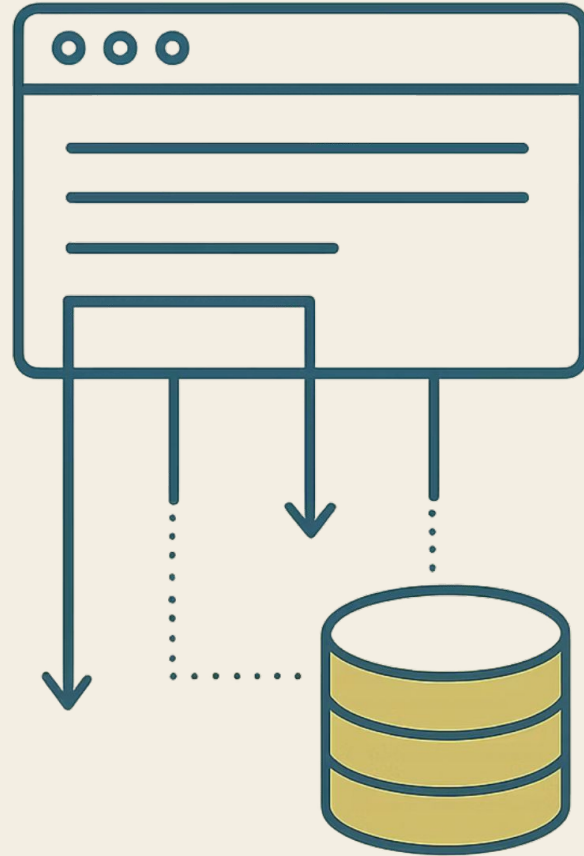
Web Scraping Tool using Python

News Headlines Extractor

Garv Kaushik - 10714812723

Sagun -10914812723

Sahil - 11514812723



What Is Web Scraping?

Web scraping is an automated technique for extracting data directly from websites — no manual copying required. It powers data analysis, academic research, and intelligent automation workflows.



Automated Collection

Fetches information from web pages programmatically



Data Analysis Ready

Structures raw HTML into usable, clean data



Our Focus

Extracting real-time news headlines from a target website

Project Objectives

This project aims to build a complete, functional pipeline — from fetching a webpage to storing structured data locally.



1

Build the Tool

Develop a Python-based web scraper from scratch



2

Fetch Real-Time Data

Connect to a live website and pull current headlines



3

Display Structurally

Show headline number, title, and link in the terminal

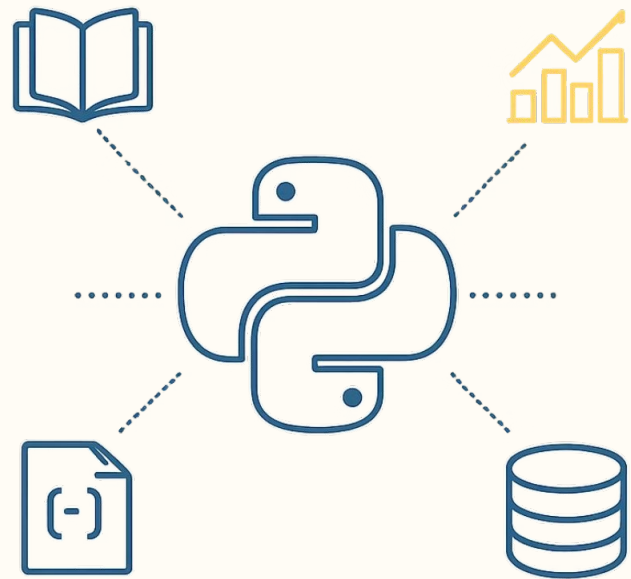


4

Export to CSV

Save all extracted data in a clean, reusable CSV file

Technologies Used



Python

The core programming language — simple, powerful, and ideal for scripting automation tasks

requests

Sends HTTP requests to fetch the raw HTML content of a webpage

BeautifulSoup

Parses and navigates HTML structure to locate specific elements like headlines

CSV Module

Writes extracted data into a structured, spreadsheet-readable file

How It Works



The scraper follows a clean, linear pipeline — each step feeds directly into the next, making the logic easy to follow, debug, and extend.

Code Logic — Simplified

```
import requests
from bs4 import BeautifulSoup
import csv

# Fetch the page
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

# Find all headlines
headlines = soup.find_all('h2', class_='title')

# Save to CSV
with open('headlines.csv', 'w') as f:
    writer = csv.writer(f)
    for i, h in enumerate(headlines, 1):
        writer.writerow([i, h.text, h.a['href']])
```

Step-by-Step Breakdown

- **requests.get()** — fetches the full webpage HTML
- **BeautifulSoup()** — parses HTML into a navigable tree
- **find_all()** — locates all headline tag elements
- **csv.writer** — loops through results and saves to file

Program Output

When executed, the tool prints a numbered list of headlines directly in the terminal — each entry includes the headline title and its source link.

Sample Output

1. *AI is Transforming Industries Worldwide* — [link]
2. *New Tech Trends Shaping 2026* — [link]
3. *Python Dominates Web Development* — [link]

What You See

- Headline number (1, 2, 3...)
- Full headline title text
- Clickable source URL



File Handling & CSV Export

Why Store Data?

Saving headlines to a CSV file makes the data persistent, portable, and ready for downstream analysis — whether in Excel, pandas, or a database.



Number

Sequential index

Headline

Full title text

Link

Source URL

Challenges & Limitations



Dynamic HTML

Website layouts change frequently — tag names and classes may shift, breaking the scraper



Internet Required

The tool depends on a live connection to fetch pages — offline use is not possible



Anti-Scraping Measures

Some sites use CAPTCHAs, rate limiting, or IP blocking to prevent automated access



Tag Identification

Finding the correct HTML tags requires careful inspection using browser DevTools

Conclusion & Future Scope

✓ What We Achieved

- Built a working Python web scraper
- Extracted real-time news headlines
- Displayed output cleanly in the terminal
- Stored structured data in a CSV file

🚀 What's Next

- Add a **Tkinter GUI** for a user-friendly interface
- Scrape **multiple websites** simultaneously
- Implement **keyword filtering** for topic-specific news
- Automate **periodic scraping** with a scheduler

THANK
YOU